

Actividad Práctica

PONDERACIÓN: 100 pts

 **Tiempo estimado: 40 minutos**

Universidad San Carlos de Guatemala
Facultad de Ingeniería.
Escuela de Ingeniería en Ciencias y Sistemas

Índice

1. MARCO FORMATIVO	2
1.1. Valores	2
1.2. Competencia(s)	2
1.3. Habilidad(es) blandas a formar	2
2. Resultado del Aprendizaje	2
2.1 Contenido	2
3. Actividad a desarrollar	3
3.1. Herramientas	3
3.2. Descripción de la actividad	3
3.3. Material de apoyo (referencias)	3
4. Rúbrica de Calificación	3
Requisitos para optar a la calificación	3
Detalle de la Calificación	3

1. MARCO FORMATIVO

1.1. Valores

Pensamiento Lógico	El estudiante es responsable de versionar su propio código, mantener un historial limpio de commits y cumplir con los entregables en el tiempo establecido.

1.2. Competencia(s)

Con la elaboración de esta actividad usted adquirirá la siguiente competencia:

Aplicar los comandos básicos de Git para inicializar un repositorio, registrar cambios mediante commits y sincronizar el proyecto con un repositorio remoto en GitHub, desarrollando buenas prácticas de control de versiones en proyectos de software..

1.3. Habilidad(es) blandas a formar

La actividad le permitirá desarrollar las siguientes habilidades:

Organización personal: el estudiante estructura y documenta su trabajo de forma ordenada. Atención al detalle: cada commit debe tener un mensaje descriptivo y claro. Autonomía: el estudiante resuelve problemas de configuración y subida de código de forma independiente..

2. Resultado del Aprendizaje

2.1 Contenido

A continuación se detalla el contenido temático que será abordado en la actividad de laboratorio.

No.	Unidad	Tema	Sub Tema
1	Estructuras de Datos Dinámicas en C#	Programación Orientada a Objetos en C#	Introducción al paradigma de POO
2	Estructuras de Datos Dinámicas en C#	Programación Orientada a Objetos en C#	Clases y Objetos (Métodos y Atributos)
3	Estructuras de Datos Dinámicas en C#	Programación Orientada a Objetos en C#	Pilares de la POO (Herencia, Abstracción, Polimorfismo, Encapsulamiento)

4	Estructuras de Datos Dinámicas en C#	Listas Enlazadas	Lista Enlazada Simple: nodos, inserción y recorrido
5	Estructuras de Datos Dinámicas en C#	Listas Enlazadas	Lista Doblemente Enlazada: navegación bidireccional
6	Estructuras de Datos Dinámicas en C#	Listas Enlazadas	Lista Circular: implementación y casos de uso
7	Manipulación XML en C#	Librerías XML en .NET	Lectura y escritura con System.Xml (XmlWriter / XmlReader)
8	Manipulación XML en C#	Librerías XML en .NET	Consultas y manipulación con XDocument (LINQ to XML)
9	Manipulación XML en C#	Librerías XML en .NET	Serialización y deserialización con XmlSerializer

3. Actividad a desarrollar

3.1. Herramientas

Git (versión 2.x o superior), Visual Studio Code o Visual Studio Community, cuenta activa en GitHub, proyecto de C# existente (el que el estudiante haya desarrollado en laboratorios anteriores), terminal de comandos (Git Bash, PowerShell o terminal integrada del IDE).

3.2. Descripción de la actividad

- El estudiante debe tomar un proyecto de C# ya desarrollado e iniciar el control de versiones usando Git. Deberá configurar Git localmente, crear un repositorio, registrar el historial de cambios con commits significativos y subir el proyecto a un repositorio público en GitHub.
- El estudiante trabajará de forma individual. Se deberá hacer un mínimo de 3 commits con mensajes descriptivos. No se permite subir el proyecto como un solo commit masivo. Se deben usar **únicamente comandos de Git por terminal** (no interfaces gráficas). La actividad debe completarse y entregarse en los 120 minutos de clase.

- Se evaluará que el repositorio sea accesible en GitHub, que el historial de commits sea coherente y progresivo, que exista un archivo README.md con la descripción del proyecto, y que el estudiante demuestre comprensión de los comandos git init, git add, git commit y git push.

3.3. Material de apoyo (referencias)

Documentación oficial de Git: <https://git-scm.com/doc> | Guía de inicio rápido de GitHub: <https://docs.github.com/es/get-started> | Pro Git Book (libre y gratuito): <https://git-scm.com/book/es/v2> | Tutorial interactivo: https://learngitbranching.js.org/?locale=es_ES

4. Rúbrica de Calificación

Requisitos para optar a la calificación

* La actividad debe realizarse y ser entregada durante el periodo de clase.

Detalle de la Calificación

Criterio	Descripción	Puntos Máximos
Implementación de Lista Enlazada	La clase Nodo y ListaEnlazada están correctamente implementadas con POO. Se pueden insertar y recorrer tickets sin errores.	30
Exportación a XML	La aplicación genera un archivo tickets.xml válido y bien estructurado con todos los campos del ticket usando al menos una librería XML.	25
Importación desde XML	La aplicación lee correctamente un archivo XML existente y carga los tickets en la lista enlazada, mostrándolos en consola.	25
Uso de múltiples librerías XML	Se utilizan al menos dos librerías XML diferentes (ej. XmlWriter y XDocument) en distintas partes de la solución.	15



Organización del código	El proyecto usa clases separadas (Ticket, Nodo, ListaEnlazada, GestorXML) con buena nomenclatura y comentarios básicos.	5
Total		100

